

ДЕНЬ 2 ИЗ 3 · 13.05.2026 · 19:00 МСК

Схема эффективного вайб-кодинга.

AI-Clone · Mastery · Business · Triggers
Артемий Муллер · СмыслоКод

Что разберём сегодня. *Слой за слоем.*

01 **Фундамент.** CLAUDE.md, .claude/, plans/, память, модели.

02 **AI-Clone.** Цифровая проекция тебя.

03 **Mastery.** Личный совет директоров.

04 **Business.** Второй мозг бизнеса.

05 **Context Engineering.** ИИ берёт только нужное под задачу.

06 **Методология вайб-кодинга.** 6 шагов от идеи до результата, промптинг, цикл реализации.

07 **Skills.** Команда экспертов за \$100/мес.

08 **MCP и CLI.** Подключение внешнего мира.

09 **Субагенты, деплой, среды.** Оркестратор, GitHub→Vercel, телефон.

10 **Triggers.** ИИ работает, пока ты спишь. Финал автоматизации.

Сегодня – **теория и методология**. Сборка ai-clone + business – в инструкции на вечер. В финале: ключевые выводы и инструкция.

Анатомия проекта. *Один взгляд - вся система.*

```
ваш-проект/
├─ CLAUDE.md           правила, навигация
├─ .claude/            настройки агента
│  └─ settings.json
│  └─ skills/          команда экспертов
│  └─ agents/          субагенты
│  └─ rules/
├─ ai-clone/           вы лично
│  └─ identity/        voice/
│  └─ principles/     feedback/
│  └─ mastery/         методы лучших
├─ business/           ваш бизнес
│  └─ products/        audience/
│  └─ economics/       execution/
├─ plans/              фичи по фазам
└─ retrospectives/     рефлексия

~/ .claude/memory/      авто-память (глобально)

— внешний мир —
MCP — серверы           YouTube, Telegram, Linear
Triggers — cron/webhook 7:00 → отчёт в Telegram
```

● **Фундамент**
CLAUDE.md · .claude/ · plans/ · retros · память. **На чём всё держится.**

● **Второй мозг**
ai-clone + business + mastery. **ИИ знает вас, бизнес, методы.**

● **Команда внутри**
Skills + субагенты + MCP. **Эксперты и связь с миром.**

● **Автономия**
Triggers. **ИИ работает, пока вы спите.**

ТА ЖЕ СХЕМА, ТЕПЕРЬ В ДВИЖЕНИИ

Как анатомия *оживает под задачу.*

01 · ВХОД

Ваша задача

«Напиши пост в моём стиле про X»



02 · CONTEXT ENGINEERING

Селектор контекста

Из **ai-clone + business + mastery** берёт **только** **нужное**. Контекст не пухнет.



03 · МЕТОДОЛОГИЯ

6 шагов вайб-кодинга

Идея → план → реализация → проверка → коммит → рефлексия.

04 · КОМАНДА

Skills · агенты · MCP

Frontend Design рисует, Bulletproof тестирует, MCP лезет наружу. Подключаются **под задачу**.



05 · РЕЗУЛЬТАТ

Готово + git commit

Фотография проекта. **Откат за 30 секунд**, если что-то не так.



06 · TRIGGERS

Отчёт в 7:00

Пока вы спите – cron запустил, сводка **в Telegram**.

| Один вход - *шесть шагов* - готовый результат. **Без вас в каждой точке.**

ГЛАВНОЕ ОБЕЩАНИЕ ДНЯ

Один и тот же ИИ. Разница в подходе.

БЕЗ СИСТЕМЫ

- 15 итераций на задачу
- Час на задачу
- Изматывает
- Объясняешь себя заново
- Генерический результат

С AI-CLONE + BUSINESS + MASTERY

- 2 итерации
- 10 минут
- С первого раза
- Контекст подгружается сам
- Готовый продукт уровня студии

К концу эфира у вас будет.

01 · ФУНДАМЕНТ

Скелет проекта

`CLAUDE.md` + `.claude/` + `plans/` – агент сам
ПОМНИТ **правила и план**.

02-04 · ЗНАНИЕ

Второй мозг

ai-clone/ – вы лично.
business/ – ваш бизнес.
mastery/ – методы лучших.

05-06 · МЕТОДОЛОГИЯ

Context · 6 шагов

Context engineering + 6 шагов вайб-кодинга + «**10 причин обосраться**».

07-08 · ИНСТРУМЕНТЫ

Skills · MCP · CLI

Frontend Design, Bulletproof и др. – **команда экспертов** и подключение внешнего мира.

09 · МАСШТАБ

Субагенты · деплой

Оркестратор → параллельные субагенты. Деплой
через **GitHub** → **Vercel**.

10 · АВТОНОМИЯ

Triggers

7:00 утра – **отчёт в Telegram**. ИИ работает, пока вы спите.

Маленький, но рабочий. *Один файл лучше неоконченного эссе.*

01

Фундамент проекта

CLAUDE.md, .claude/, plans/, память, модели - единая система, на которой всё держится

CLAUDE.md

– *шаблон*, который ставите в первый день.

{название проекта} - навигация 1

Этот файл - системный промпт.
Claude читает его ПЕРЕД каждым сообщением. Без исключений.

Что это

{Что делаем, для кого, зачем - 2-3 предложения.}
{Платформа: лендинги, личный кабинет, админка, оплаты, бот.}

Стек 2

- Next.js 14 · TypeScript · Tailwind CSS
- Prisma + PostgreSQL · Redis
- Деплой: Coolify

Структура 3

```
project/
├─ CLAUDE.md           ← правила, навигация (этот файл)
├─ ai-clone/           ← вы лично
│   ├─ identity/       ценности, биография
│   ├─ voice/          тон, словарь
│   ├─ principles/     фреймворки решений
│   └─ feedback/       правила через ошибки
├─ business/           ← бизнес-контекст
└─ products/
```

⚡ СКРОЛЛ

- 1 Описание.** Что за проект, для кого. 2–3 предложения, чтобы агент сразу попал в контекст.
- 2 Стек.** Языки, БД, деплой. Без него агент гадает и предлагает «как принято в React-проектах».
- 3 Структура.** Карта офиса для нового сотрудника – куда идти за каким файлом.
- 4 Роутинг.** Главная защита от распухания: CLAUDE.md = оглавление, детали – в отдельных файлах.
- 5 Правила.** Поведенческий договор. Без него – стажёр. С ним – инженер.
- 6 Язык.** Иначе агент скатывается в английский на длинных диалогах.

До 150–200 строк. Команда `/init` сгенерирует базовый автоматически – дальше дорабатываете.

Пять блоков. *Карта офиса* для сотрудника.

- 01 **Описание проекта.** Что делаем, для кого, зачем. 2–3 предложения.
- 02 **Технологический стек.** Языки, фреймворки, библиотеки. Без этого – каша.
- 03 **Структура файлов.** Где что лежит. Карта офиса для сотрудника.
- 04 **Правила и стиль.** Формат кода, ограничения, предпочтения.
- 05 **Роутинг на файлы.** Ссылки на детальные документы. CLAUDE.md = оглавление, не книга.

Пихаете всё в один CLAUDE.md - контекст распухает, лимиты улетают за час. Поэтому роутинг.

ПРАВИЛА, КОТОРЫЕ МЕНЯЮТ ВСЁ

CLAUDE.md = *трудовогой договор* агента.

Без правил – стажёр. С правилами – инженер. Шесть пунктов, которые ставлю в каждый проект:

- 01 До 200 строк.** Больше – контекст сжирается. Детали выноси в `business/` и `.claude/rules/`.
- 02 Заканчивай отчётом.** «Каждый диалог завершай: какая задача, как решал, было → стало».
- 03 Коммит после задачи.** Коммит = фотография проекта. Сломали? `git` откатит за 30 секунд.
- 04 Всегда Bulletproof.** «Когда ставлю задачу на фичу – всегда используй `skill bulletproof`».
- 05 Сверяйся с business.** При текстах – `audience/avatar.md` . При UI – `assets/brand-guidelines.md` .
- 06 Безопасность.** Запрещай в `settings.json` : `rm -rf` , `curl + export` , доступ к `.env` .

Папка

.claude/

- .claude/
- └ settings.json - настройки, доступы, режимы
 - └ skills/ - установленные навыки
 - └ agents/ - субагенты
 - └ rules/ - дополнительные правила

ЛОКАЛЬНАЯ · .CLAUDE/

Внутри проекта

Только для этого проекта.

ГЛОБАЛЬНАЯ · ~/.CLAUDE/

Для ВСЕХ проектов сразу

Ваши универсальные скиллы, правила, память.

Что запрещать в `settings.json`

Bypass Permissions = полная автономия агента. **Без запретов** – он может натворить.

→ `rm -rf *` – **удаление диска**

→ `export + curl` – **утечка секретов**

→ Доступ к `.env` – **API-ключи, пароли**

→ `wget` с переменными – **экспорт данных**

*Запрет в `settings.json` – **агент физически не сможет это сделать**, даже если кто-то его попросит.*

ХУКИ

Звуковые *уведомления*.

ПРОБЛЕМА

Запустили задачу - ушли

Claude закончил через 2 минуты. Вы вернулись через час. Час простоя.

РЕШЕНИЕ

Хук = звуковой сигнал

При завершении задачи или запросе разрешения. Слышно из другой комнаты.

ПРОМПТ

«Настрой хук для звукового сигнала при завершении диалога и при запросе разрешений.»

Авто-память `~/.claude/memory/` — агент пишет правила за вас.

01 **Вы говорите:** «не хардкодь даты когорт, всегда бери из базы».

02 **Агент записывает** в файл `feedback_no_hardcode_cohort.md` в папку `memory/`.

03 **В каждом следующем диалоге** агент сам это правило вспоминает. Вы ничего не напоминаете.

У меня в проекте уже 28+ **файлов памяти** - все мои правила, предпочтения, ссылки на SSH, Coolify, бренд. **Один раз сказал - агент помнит всегда.**

Мозг проекта: *5 частей + машина времени.*

CLAUDE.md

Правила проекта и навигация. Агент читает первым перед каждым действием.

ai-clone/ + business/

Второй мозг: вы + продукт + аудитория + цены + цели + экономика.

plans/

Технические планы по фазам. **Один план = одна функция**, статус видно сразу.

retrospectives/

Рефлексия после каждой сессии: что сделали, что запомнить.

live-metrics.md

Маркер **LIVE** в тексте = агент идёт в базу за свежими цифрами, не хардкодит.

git сверху всего

Машина времени. Откатил, сравнил, вернулся к любой точке без страха.

*Пять папок плюс git - у проекта появляется **память**, которой у вас раньше не было.*

Один план – *одна функция.*

- 01 **Один план = одна функция.** Каждая крупная фича – свой файл. Имя: `YYYY-MM-DD-название.md` . Не мешайте в кучу.
- 02 **Фазы и чек-боксы.** План делится на Фазу 1, Фазу 2... У каждой статус `[ ]` или `[x]` . Можно передавать разным агентам.
- 03 **Разбиение для параллели.** «Раздели на фазы с максимальной параллельностью». Одна блокирующая → 5 параллельных агентов.

План в репозитории, не в чате. Любой агент подхватит в любой момент.

Plan Mode *vs* папка plans/

PLAN MODE

- Живёт только в текущем диалоге
- Закрыл окно - потерял план
- Нельзя передать другому агенту
- Нет версий и истории
- Ок для блиц-задач на 10 минут

ПАПКА PLANS/

- Файл - план живёт в репозитории
- Любой агент подхватит в любой момент
- Фазы, чек-боксы, статусы [] / [x]
- История правок - через git
- Базовый инструмент серьёзной работы

Plan Mode - для коротких идей. Папка plans/ - для всего, что длится дольше одного диалога.

Контекстное окно – оперативная память агента.

В контекст при **каждом** сообщении: `CLAUDE.md` + описания скиллов + MCP-инструменты + история диалога + файлы через `@`.

1 токен $\approx$ 0.75 слова. Окно: **200K** (Sonnet) / до **1M** (Opus).

CONTEXT ROT

- Больше 50–60% заполнения
- Качество падает
- Лимиты улетают
- Агент «тупеет»

КОМАНДЫ УПРАВЛЕНИЯ

- `/context` – посмотреть
- `/compact` – сжать
- `/clear` – очистить
- `/rewind` – откат

Золотое правило: 1 задача = 1 диалог.

КТО ЗА ЧТО ОТВЕЧАЕТ

Модели: *Opus / Sonnet / Haiku*.

АРХИТЕКТОР

Opus 4.6

Глубокое мышление, архитектура, анализ. До **1M** контекста. Самый дорогой.

→ планирование, сложные решения

ИСПОЛНИТЕЛЬ

Sonnet

Баланс скорости и качества. Код по готовой спецификации.

→ реализация, правки, написание кода

БЫСТРЫЙ

Haiku

Самая быстрая и дешёвая. Простые задачи, большие объёмы.

→ рутина, обработка данных

Лайфхак: `/model → "opus plan"` - Claude сам выбирает модель. **Thinking Max + ultrathink** для сложных задач.

02

AI-Clone

Цифровая проекция тебя

ОПРЕДЕЛЕНИЕ

AI-Clone – это не «ИИ, который копирует стиль».

Это ИИ, который знает тебя глубоко – **на уровне принятия решений.**

Что ценишь

Что отвергнешь сразу

Принципы в работе

Темп · амбиции · вкус

Цифровая проекция. Думает как ты. Сам находит проблемы. Сам приносит решение.

Ты говоришь «да» – он делает весь цикл.

ПОЧЕМУ «КЛОН», А НЕ «СО-ФАУНДЕР»

Сначала был *AI-Co-Founder*. Это слишком плоско.

СО-ФАУНДЕР

Сидит в чате. Помогает.

Реактивен. Отвечает, когда спрашиваешь.

КЛОН

Твоё мышление, перенесённое в
файлы.

Проактивен. Живёт в файловой системе, читается ИИ всегда.

ИИ работает не *вместо* тебя и не *за* тебя.
А *как* ты.

Один *AI-Clone*. Много проектов.

AI-Clone живёт глобально. Каждый проект имеет свой `business/` . Подцепляются через `CLAUDE.md` .

```
ai-clone/                ← один на все ваши проекты

project-a/
  business/              ← карта этого бизнеса
  CLAUDE.md              ← ссылка на ai-clone

project-b/
  business/
  CLAUDE.md              ← та же ссылка на ai-clone
```

| *Вы - один человек. Клон - один. Проектов может быть много.*

Структура

ai-clone/

ai-clone/

— INDEX.md	- точка входа, ИИ читает первой
— role.md	- кто я: бэкграунд, что важно
— identity/	- values, vision, mission, biography
— voice/	- tone, vocabulary, стоп-слова
— thinking/	- mental-models, как принимаю решения
— principles/	- product, code, business
— feedback/	- правила, выученные через ошибки
— style/	- telegram-format, стоп-слова
— mastery/	- чужое мастерство (отдельный блок 03)
— reference/	- инструменты, без секретов

Каждая папка - один аспект тебя.

Каждый файл - один вопрос: «кто я в этом измерении?».

identity/ — *кто ты.*

values.md	- что я ценю
vision.md	- куда я иду
mission.md	- зачем я здесь
biography.md	- что меня сделало

Не «о себе» из резюме. **Вертикальный срез:** ценности, направление, миссия, путь.

*ИИ читает это **первым** - и весь дальнейший разговор проходит через эту линзу.*

`voice/` — *как ты говоришь.*

`tone.md` - энергия речи: дерзко / спокойно / тепло
`vocabulary.md` - твои слова: что используем, что не используем
`stop-words.md` - что палит ИИ: длинное тире, «не А, а Б»

Без `voice/` ИИ пишет за тебя **усреднённый русский**.

С `voice/` ИИ пишет твоими словами, *твоим ритмом, твоим уровнем дерзости.*

`principles/` — *твои правила игры.*

<code>product.md</code>	- как ты строишь продукты
<code>code.md</code>	- как ты пишешь код (если пишешь)
<code>business.md</code>	- как ты ведёшь бизнес

Не «лучшие практики из интернета». **Твои принципы, выработанные ошибками и победами.**

Не делать на выброс

Качество > скорость

Никаких костылей

Минимализм + дерзость

*ИИ читает - и предлагает решения **в твоей логике.***

feedback/ – *самообучение.*

Главная папка. Куда складываются правила, **выученные через ошибки.**

Артемий

не пишет правила руками – говорит **«не так, потому что...»** в момент ошибки.

ИИ

оформляет это в файл по канону: **Правило → Почему → Как применить.**

Через месяц

десятки правил, которые вы **не повторили дважды.**

КАК НАПОЛНИТЬ

5 способов наполнить `ai-clone/`

- 01 **Голосовое интервью с ИИ.** Главный способ. Сегодня соберём вживую.
- 02 **Транскрипции эфиров и Zoom-звонков.** Если ведёшь публичную деятельность – у тебя уже тонна материала.
- 03 **Email-переписка.** Деловая. Скормить за месяц-два.
- 04 **Авто-память ChatGPT.** Маленькая, но кое-что вытащить можно.
- 05 **Telegram-чаты.** Технически можно. Но я бы не стал – там слишком много шума.

Промпт для сборки `ai-clone/` через интервью

ПРОМПТ

«Ты интервьюер. Соберёшь мой профиль для AI-Clone.

Задай мне вопросы по блокам:

- identity (кто я: ценности, путь, миссия)
- voice (как я говорю: тон, словарь, что не говорю)
- principles (мои правила в продукте, бизнесе, работе)
- thinking (как принимаю решения, какие фреймворки)
- style (что отвергаю, какая эстетика)

По 5-7 вопросов на блок. По одному вопросу за раз.

Предлагай 2-3 варианта ответа - я выберу или дополню.

После каждого блока сохраняй ответы в соответствующие файлы в ai-clone/.

Говорю голосом через /voice - не печатаю.»

Скорость - в 10 раз выше, чем писать с чистого листа. За вечер собирается базовая версия клона.

С клоном *vs* без.

БЕЗ AI-CLONE

- Каждый чат – объясняешь себя заново
- ИИ предлагает шаблонные решения
- Час уходит на то, чтобы привести к нужному

С AI-CLONE

- ИИ сам предлагает то, что ты бы и так выбрал
- Отсекает шлак до того, как ты его увидишь
- Работает с твоим уровнем требований, не со стандартным

| Но даже клон одного - недостаточно. Нужно ещё знание области. *Это мастерство.*

03

Mastery

Личный совет директоров для ИИ

РАЗЛИЧИЕ

Скилл *vs* мастерство.

СКИЛЛ

Навык

- «Сделай PDF»
- Одно умение
- 1-3K токенов

Скилл – **инструмент**.

МАСТЕРСТВО

Глубокое понимание

- «Как пишут книги, которые продают»
- Накопленный опыт лучших умов
- Концентрат целой области

Мастерство – **точка зрения эксперта**.

ГЛАВНЫЙ ПРИНЦИП MASTERY

Не книги.

Не пересказ.

Не саммари.

У каждого великого автора - 1-2-3 гениальных метода. Достаточно.

Вся их жизнь - чтобы выделить пару методов. Мы берём только их.

Объединяем в одну экосистему. Получается **концентрат лучших умов мира**, а не пересказ ютуба.

Как собрать *одну папку* mastery.

- 01 **Выбираем область.** Например, «копирайтинг».
- 02 **Находим 3–5 авторов с реальной экспертизой.** Огилви, Эл Райс, Кэплз, Шугерман.
- 03 **Из каждой книги – только методы.** Не пересказ. Не цитаты. Метод как пошаговый алгоритм.
- 04 **Складываем в** `mastery/copywriting/` . Один автор = один файл.
- 05 **Связываем викилинками.** `[[ogilvy-headlines]]` , `[[caples-tested]]` .

Результат: `mastery/copywriting/` – ~15 KB концентрата. *Не 5 книг по 200 страниц.*

ПРИМЕР ИЗ МОЕЙ СИСТЕМЫ

Мои папки

mastery/

```
mastery/
├─ youtube/           - рост, монтаж, упаковка
├─ copywriting/       - заголовки, оффер, лендинги
├─ email-marketing/   - открываемость, последовательности
├─ decision-making/   - Мангер, инверсия, ментальные модели
├─ negotiations/      - Босс, Карасс
├─ product/           - Jobs-to-be-Done, Сagan
├─ hiring/             - структурное интервью
├─ public-speaking/   - структура выступлений
└─ financial-analysis/ - юнит-экономика, прогноз
```

| Каждое - *концентрат лучших умов* по теме. У вас будет своя комплектация.

Запрос: «что у нас по цифрам в этом квартале?»

- 01 Видит ключи «цифры», «квартал»
- 02 Идёт в mastery/ - находит financial-analysis/
- 03 Идёт в business/economics/ - берёт данные
- 04 Идёт в business/goals/ - берёт цели квартала
- 05 Применяет методы из mastery к данным из business

Возвращает разбор цифр через линзу эксперта по финанализу. Не «просели заявки» - а «юнит-экономика дрейфует, поднимай LTV...».

ЭФФЕКТ

Mastery превращает обычного агента в команду экспертов под рукой.

УТРОМ

Копирайтер

ЧЕРЕЗ ЧАС

Переговорщик

ВЕЧЕРОМ

Финансовый аналитик

Один ИИ. Десятки экспертных линз.

Это и есть *context engineering* в чистом виде.

04

Business

Второй мозг твоего бизнеса

ПРОСТОЕ ОПРЕДЕЛЕНИЕ

Папка на компьютере, где живёт весь ваш бизнес.

Простыми словами: **папка с текстовыми файлами**. Не в голове. Не в Notion. Не в чатах. **Рядом с кодом**, в одной папке с проектом.

✓ Голова разгружается

–
перестаёт
держаться
200
мелочей

✓ ИИ читает файлы

и
сразу
понимает
бизнес

✓ Через полгода

открываете
файл
и
помните,

Обычно бизнес живёт в 4 местах. *Везде кусок.*

ЧТО ЛОМАЕТСЯ

- Через месяц забыли, почему подняли цену
- Новый человек спрашивает по часу
- ИИ пишет «качественные решения» – не знает вашего бизнеса
- В голове одновременно 50 нерешённых вопросов

ЧТО ДАЁТ ВТОРОЙ МОЗГ

- Одно место, где лежит всё
- ИИ работает как сотрудник на 2-м году
- На любой вопрос «а почему мы...» – ответ из файла
- Голова свободна. Решения быстрее.

Структура

business/

business/

- | INDEX.md - карта папки + правила + роутинг
- | products/ - что продаёте, за сколько
- | audience/ - для кого, их боли
- | economics/ - выручка, расходы, юнит-экономика
- | marketing/ - каналы, воронка, контент
- | processes/ - как устроена работа
- | goals/ - годовые, квартальные, месячные

*Каждая папка - одна тема. Каждый файл - один вопрос. **Ничего лишнего.***

На **тренинге** раскрываем подузлы: аватар, сегменты, путь клиента, юнит-экономика, воронка, копирайтинг, отзывы, бренд. На практикуме - укрупнённая структура.

«У меня же есть *Notion*». Чем это отличается?

Я пробовал всё. У них одна проблема: **они отдельно от работы**. ИИ туда не заглядывает.

NOTION / GOOGLE DOCS

- ИИ туда не заглядывает
- Копируешь руками
- Красивые заметки для себя
- Живёт отдельно от кода
- История через версии Notion

ПАПКА BUSINESS/

- ИИ читает каждый раз
- Подгружается само
- Рабочий инструмент ИИ
- Живёт рядом с кодом
- История через git

ТОНКИЙ МОМЕНТ, КОТОРЫЙ МЕНЯЕТ ВСЁ

У большинства: *мозг* и *код* в разных местах. У нас – рядом.

```
project/
├─ ai-clone/    ← ты
├─ business/    ← бизнес
├─ src/         ← код
└─ CLAUDE.md    ← правила
```

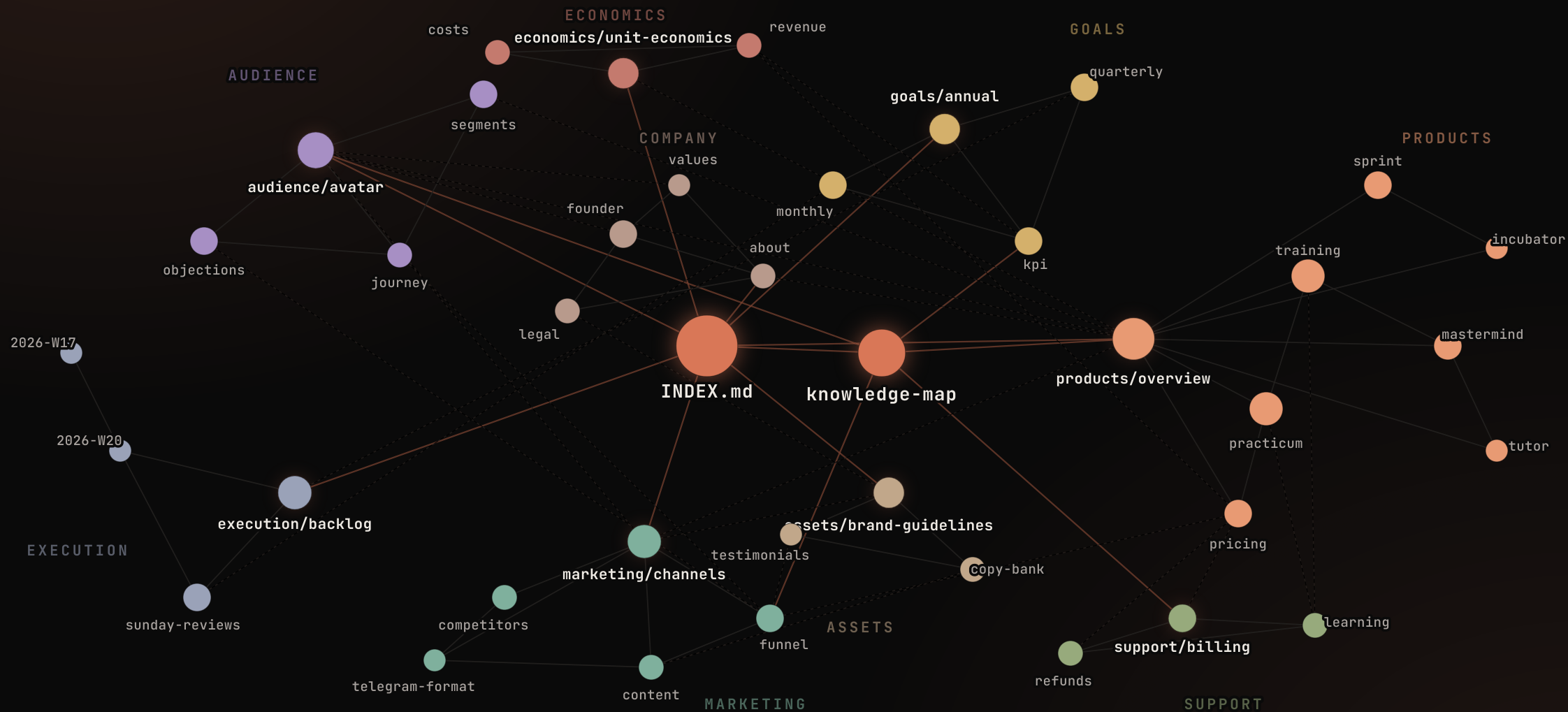
ИИ читает *три мира одновременно*. Без копинасты.

Память ИИ – *как у человека.*

- 01 **Короткая.** Текущий диалог. Закрыв чат – стёрлось.
- 02 **Рабочая.** CLAUDE.md + файлы через @. То, что ИИ держит перед глазами весь диалог.
- 03 **Долгая.** ai-clone + business + plans + git. Подгружается по требованию.

*Задача мозга - чтобы первые два слоя **не перегружались**, но имели доступ ко всему.*

Папка `business/` – *единый организм*, а не папка с файлами.



Каждый файл – *узел*. Каждая `[[ссылка]]` – *ребро*. ИИ читает *связи целиком*, а не пятна.

Викилинки – *граф знаний*.

Аватар клиента

Типичные боли см. `[[objections]]`

Путь покупки: `[[journey]]`

Бренд-голос: `[[brand-guidelines]]`

Каждая `[[ссылка]]` – узел графа. Obsidian рендерит визуально.

*Ваш бизнес – **единый организм**, а не набор файлов.*

*Поставьте Obsidian, откройте `business/` как *vault* – увидите граф за 10 секунд.*

Промпт для сборки `business/`

ПРОМПТ

«Ты интервьюер. Задай мне вопросы по блокам: аудитория, продукты, экономика, цели.

По 5-7 вопросов на блок. По одному вопросу за раз.

Предлагай варианты ответа, чтобы я мог выбрать, а не писать с нуля.

После каждого блока сохраняй ответы в соответствующие файлы в `business/`.»

Секретная фраза: «Предложи варианты сам». Снимает паралич чистого листа.

Плюс голосом через `/voice` – говорите быстрее, чем печатаете. Brain dump за 3 минуты = **готовое ТЗ**.

Мозг жив, *пока в него докладывают.*

- 01 **В течение недели.** Накапливаются вопросы, идеи. Скидываю голосом или текстом.
- 02 **В конце недели.** «Вот накопленное, разбери и обновь файлы в business/». ИИ сам находит, куда положить.
- 03 **После каждой сессии.** Рефлексия в retrospectives/. 30 секунд – накапливается на годы.
- 04 **После запуска потока.** «Проанализируй вопросы эфира и обновь objections.md».

Мозг растёт *сам, от работы.* Вы не тратите на это отдельные часы.

Цифры устаревают. *LIVE* – лекарство.

ОБЫЧНЫЙ ПОДХОД

Хранить значение

- Выручка за март: 2,4 млн
- Учеников: 1000
- Через месяц – протухло

С LIVE-МАРКЕРОМ

Хранить указатель

- Выручка за март: ****LIVE****
- → выполни SQL из live-metrics.md
- Цифра всегда свежая

В файлах `business/` стоит метка ****LIVE****. ИИ видит метку → идёт в базу → получает свежие данные.

Файлы не просто данные – *они диктуют правила ИИ.*

В Notion – пассивный справочник. У нас – **активное правило**. `CLAUDE.md ≤ 200 строк`, загружается полностью в контекст. Детали – в роутинг.

→ **ОБЯЗАТЕЛЬНО** при UI-работе сверяйся с `brand-guidelines.md`

→ **НИКОГДА** не хардкодить даты и цены – всегда из базы

→ Все письма – через единый `layout.ts`

→ Коммитить только свои изменения – **никогда** `git add -A`

→ Перед push – `npm run build`, Coolify падает на ESLint

→ **НИКОГДА** не делай deploy без явной команды

*Мозг не подсказывает. Он **заставляет**. ИИ физически не может сделать по-другому.*

После Потока 1 пришло *1000* *вопросов.*

Чат эфиров, поддержка, формы. Что сделано **за один день**:

- 01 Выгрузил все 1000 вопросов в один файл
- 02 ИИ разложил по типам, нашёл паттерны
- 03 Обновил `audience/objections.md` - новые возражения
- 04 Обновил `audience/avatar.md` - уточнил боли
- 05 Написал план улучшений продукта
- 06 Я одобрил - ИИ внёс правки в ЛК, письма, автоматизации

Команда из 10 человек не сделала бы это за месяц. Один человек + система - один день.

Второй мозг нужен *не агенту*. Он нужен *вам*.

до

- Голова забита мелочами, сон плохой
- На вопрос «а какая маржа?» – 40 минут в таблицах
- Каждый новый человек – пересказ с нуля
- Через полгода не помните, почему так решили
- 20 минут копипасты в каждый чат

после

- Голова чистая, решения – быстрые
- На любой вопрос – ответ из файла за секунды
- Новый человек читает `INDEX.md` и через час в теме
- Прошлые доступно в любой момент
- Запустили задачу – агент читает мозг и делает

| *Агент – просто инструмент, чтобы извлечь из головы то, что годами копилось.*

САМОЕ НЕДООЦЕНЁННОЕ СВОЙСТВО

Спросить *своё прошлое* – за 3 секунды.

«Почему отказались от рассрочки в марте?»

→ ответ из `retrospectives/`

«Какие возражения были перед Потоком 2?»

→ `audience/objections.md`

«Что мы пробовали в маркетинге?»

→ `marketing/channels.md`

«Сколько заработали в феврале?»

→ **LIVE**-запрос к прод-базе

«Как формулировали оффер?»

→ `assets/copy-bank.md`

Плюс `git blame` – видно когда и почему. *Ни один человек так не умеет.*

05

Context Engineering

Как ИИ берёт из всех файлов только нужное под задачу

ЭВОЛЮЦИЯ

Промпт уходит. Контекст - приходит.

2023

Промпты

«Напиши пост в стиле X, длиной Y»

2024

Золотые промпты

*Многосоставные конструкции, шаблоны,
библиотеки*

2026 · СЕЙЧАС

Context Engineering

*Промпт короткий. Контекст в файлах. ИИ сам
выбирает.*

| Не «как написать промпт». А «как устроить файлы, чтобы промпт стал не нужен».

ЧТО ЗНАЧИТ «ПРАВИЛЬНЫЙ»

Не «ИИ знает всё». А *«берёт только то, что надо под задачу»*.

Запрос: «что у нас по выручке?» ИИ собирает контекст:

01 `ai-clone/principles/business.md` ← как ты смотришь на деньги

02 `mastery/financial-analysis/` ← метод эксперта

03 `business/economics/revenue.md` ← данные

04 `business/goals/quarterly.md` ← цели

05 `CLAUDE.md` → LIVE-маркер → SQL ← свежие цифры

Не «весь бизнес целиком». Не «все файлы скопом». Только то, что нужно для этой задачи.

5 слоёв в *каждом новом окне.*

01 `CLAUDE.md` – **правила проекта**, навигация

02 `ai-clone/` – **кто ты**, как думаешь

03 `mastery/` – **экспертная линза** под задачу

04 `business/` – **что мы делаем**

05 **История диалога** – текущий разговор

«Когда ты открываешь новое окно – у тебя *уже под капотом всё,*
что тебе надо».

Главная ошибка – запихнуть всё.

10 часов Zoom-расшифровки → ИИ путается → берёт не то → лимиты улетают. **Решение:** каждый файл – сжатый концентрат.

01 **1 задача = 1 диалог.** Не пытайтесь делать несколько разных вещей в одном чате.

02 **Контекст ≤ 60%.** Больше – `/compact` или `/clear`.

03 **Ничего не делай руками.** Всё, что можно сделать через ИИ – делай через ИИ.

Меньше – лучше. *Но точнее.* Дальше – как мы работаем с этим в повседневности: **методология вайб-кодинга.**

06

Методология вайб-кодинга

6 шагов от идеи до результата + промптинг + цикл реализации

От идеи к *CLAUDE.md*.

- 01 Инструкция.** `instructions.md` – человеческим языком. Можно голосом (`/voice`). Brain dump за 2 мин = ТЗ.
- 02 Уточняющие вопросы.** Plan Mode. @ файл + «Задай вопросы, пока не уверен на 95%». Не знаете – «на твоё усмотрение».
- 03 Создание CLAUDE.md.** Claude создаёт: стек, дизайн-систему (цвета, шрифты), структуру, соглашения. Проверяете.

Эти три шага – до кода. Самые важные. Ошибка тут = день переписки.

От плана к *реализации*.

04 **Реализация (новый диалог!).** Чистый контекст. Claude читает `CLAUDE.md`. Bypass Permissions. «Приступай». To-do list с галочками.

05 **Итерации.** Localhost → смотрите → говорите просто: «Поменяй шрифты», «Добавь анимацию». `/rewind` если не туда.

06 **Чанки для больших фич.** План с блоками. Каждый блок = отдельный диалог. Контекст не переполняется.

Новый диалог на этапе 4 - критично. Иначе контекст разогрева подаст ошибки реализации.

Хороший *vs* плохой промпт.

ПЛОХОЙ

«Сделай сайт для выгула собак»

Результат: генеричный, безликий, без деталей.

ХОРОШИЙ

«Лендинг Harry Paws. Hero. 3 услуги с ценами. Форма. Синяя схема.»

Результат: конкретный, структурированный.

- Начинайте в Plan Mode – пусть Claude спланирует
- «Задай вопросы, пока не уверен на 95%»
- Конкретно, но не перегружая. Голосовой ввод быстрее печати
- Скриншот-референс > 500 слов описания

Скриншот > *500 слов описания.*

01 **F12 → Ctrl+Shift+P** → capture full-size screenshot

02 **Перетащите** скриншот в Claude Code

03 **Скажите:** «Сделай в похожем стиле, но с нашими цветами»

Бонус: скопируйте CSS-стили (**F12 → Elements → Styles**) и дайте вместе со скриншотом. Claude воссоздаст стиль с высокой точностью.

Скриншоты *в буфер.*

🍏 MAC

- `Cmd + Shift + 4` - на рабочий стол
- `Cmd + Shift + Ctrl + 4` - в буфер
- Settings → Keyboard → Screenshots - перенастрой хоткей
- Работай с буфером: `Ctrl+V` → в чат

🪟 WINDOWS

- `Win + Shift + S` - открывает Snipping Tool
- Выделил область → в буфере
- `Ctrl + V` прямо в чат Claude
- Всё из коробки, настраивать нечего

Голос (*Wispr*) + скриншот-в-буфер = *тандем, ускоряющий работу в 3-4 раза.*

Research → Plan → Critique → *Phases* →
Implement → Review.

- 01 **Deep Research.** 5 параллельных агентов. Анализ аналогов, данных, рынка. Выжимка в файл.
- 02 **План.** Агент на основе research пишет план: архитектура, решения, технологии.
- 03 **Критика.** Отдельный агент-критик ищет дыры: «что не учли? где обосрёмся?»
- 04 **Фазы.** План делится на Фазу 1, 2, 3... с максимальной параллельностью.
- 05 **Реализация.** Каждая фаза – отдельный субагент. Чистый контекст. Параллельно.
- 06 **Review.** Security Review + поиск ошибок. Агент-проверяющий смотрит итог свежими глазами.

*Research → Plan → Critique - до кода. Ошибка на плане = 1 минута.
Ошибка в коде = день переписки.*

«10 причин *обосраться*».

ПРОМПТ

«У меня эфир / запуск / деплой. Найди 10 причин, из-за которых мы обосрёмся. Покритикуй жёстко.»

Что даёт

Security Review + логические дыры + пропущенные кейсы.

Зачем нужен

Claude находит то, что **вы не увидите сами**.

Применяйте на *каждый план и каждый релиз*. Самые дорогие ошибки поймаете до прода.

To-do list + *скриншот-цикл.*

TO-DO LIST

Список задач с галочками

Видите прогресс, можете уйти и вернуться. Усиление: «не переходи к следующему пункту, пока не уверен на 95%».

СКРИНШОТ-ЦИКЛ

Claude смотрит на результат глазами

«После создания каждой страницы сделай скриншот через Puppeteer, проверь визуально, исправь. Минимум 2 прохода».

Claude Code – не чёрный ящик. *Будьте искренне любопытны*: «что ты сделал?». За неделю поймёте 80%.

Семь правил, *которые экономят часы.*

- 01 **Один чат = одна задача.** Закончил – закрой чат, открой новый. Очищает контекст и голову.
- 02 **Контекст ≤ 60%.** Больше – `/compact` или `/clear`. Иначе агент тупит, делает не то.
- 03 **Ничего не делай руками.** Папки, файлы, команды – всё через Claude. Ты управляешь, не исполняешь.
- 04 **Ошибки чини в том же чате.** Не открывай новый чат для дебага – потеряешь контекст.
- 05 **Долго думает – жди.** 5–10 минут нормально. Не перезапускай. Совсем висит – `/clear` и короче.
- 06 **Каждая фича = план.** В `plans/` с фазами `[ ]/[x]`. И рефлексия в `retrospectives/`.
- 07 **Выучи как мантру.** Разница между «бьётся 5 часов» и «делает за 30 минут».

07

Skills

Суперсила Claude Code: команда экспертов за \$100 в месяц

ОПРЕДЕЛЕНИЕ

Скилл = *папка с инструкцией.*

Markdown-инструкция + опциональные файлы (скрипты, шаблоны, примеры). **Узкая экспертиза в одной задаче.**

БЕЗ СКИЛЛОВ

Стажёр общего профиля

Сделает, но средне. AI-slop.

СО СКИЛЛАМИ

Команда экспертов

Каждый скилл = специалист. По задаче.

Аналогия: *повар без рецепта vs повар с рецептом от шеф-повара.*

Как устроен скилл + *прогрессивная загрузка.*

SKILL.MD

```
---  
name: pdf-creator  
description: Создаёт PDF-документы  
---
```

```
# PDF Creator  
## Шаг 1: Собрать данные  
## Шаг 2: Создать макет  
## Шаг 3: Сгенерировать
```

Ур. 1 Читает `name + description` всех скиллов. ~100 токенов на скилл.

Ур. 2 Нашёл подходящий - загружает полный `skill.md` .1–3K токенов.

Ур. 3 Доп. файлы (скрипты, референсы) по необходимости. По ситуации.

СОДЕРЖИМОЕ

Что бывает *внутри* скилла.

`skill.md`

Главная инструкция. Пошаговые действия.

`scripts/`

Python, Shell, JS скрипты. Скилл PDF = Python-скрипт генерации.

`references/`

Примеры, эталоны. Ваш tone of voice, примеры постов.

`templates/`

Готовые заготовки документов, страниц, отчётов.

В `skill.md` – ссылки (пути) на эти файлы. Файлы могут быть где угодно, главное – правильный путь.

Как *вызвать*. Как *установить*.

СЛЭШ - КОМАНДА

/pdf /excel /frontend-design

Прямой вызов конкретного скилла.

ЕСТЕСТВЕННЫЙ ЯЗЫК

«Создай мне PDF-отчёт»

Claude сам найдёт подходящий скилл.

МАРКЕТПЛЕЙС

skills.theanage

Установка в 2 клика, сортировка по популярности.

МАРКЕТПЛЕЙС

smithery.ai

Категории: research, design, code.

OFFICIAL

Anthropic

PDF, XLSX, PPTX, Frontend Design, Skill Creator.

СВОИ

Skill Creator

Через Skill Creator или вручную.

MUST-HAVE

Скиллы, *без которых нельзя.*

ОБЯЗАТЕЛЬНО

Frontend Design

Кардинально улучшает дизайн. Без него AI-slop, с ним – **профессионально.**

ОБЯЗАТЕЛЬНО

Skill Creator

Мета-скилл от Anthropic. **Создаёт, тестирует, оптимизирует** другие скиллы. Eval.

РЕКОМЕНДУЕМ

PDF / XLSX / PPTX

Документы, таблицы, презентации прямо из Claude Code.

Не устанавливайте от непроверенных – *бывают вредоносные.* Проверяйте `skill.md` перед установкой.

Аудит скилла *перед установкой.*

ПРОМПТ-АУДИТ

«Скачай исходники [URL]. Прочитай README, SKILL.md, все .md и .sh. Проверь на опасные операции: чтение .env, отправку на внешние URL (curl/wget/fetch), удаление файлов (rm/unlink), запуск бинарников. Дай вердикт: безопасно / подозрительно / опасно.»

Правило: Мало звёзд + неизвестный автор = не ставим. Каждый скилл проходит этот промт **до установки**. Ставим локально в проект (`.claude/skills/`), не глобально.

Скилл работает под твоим токеном в твоей системе. Прокидывать туда чужой код без аудита = сдать ключи от квартиры незнакомцу.

Шесть шагов *к своему скиллу.*

01 **Имя и триггер.** Как называется? Какие слова активируют?

02 **Цель.** Одно предложение: что на выходе?

03 **Пошаговый процесс.** Что, в каком порядке, какие решения.

04 **Референсы.** Brand guidelines, примеры, документация API.

05 **Правила.** Что может пойти не так? Guardrails.

06 **Цикл улучшения.** Как улучшается после каждого запуска.

Лайфхак: сделайте задачу с Claude, понравился результат – «Превратим в скилл. Задай вопросы».

Цикл *обратной связи*.

1

ВЫЗВАЛИ

Запустили скилл

2

ПОСМОТРЕЛИ

Оценили результат

3

СКАЗАЛИ

«что не так»

4

SKILL.MD

обновился

5

СНОВА

Лучше каждый раз

1 - 2 Запуски: средний результат, нужны правки

3 - 5 Запуски: заметно лучше, основные проблемы решены

10+ Запуски: **стабильно отлично каждый раз**

ЭКОНОМИКА

Команда экспертов *за \$100 в месяц.*

Подписка Max – \$100/мес. Получаете: **дизайнера, верстальщика, копирайтера, аналитика, генератор презентаций и PDF.**

ФРОНТЕНД-РАЗРАБОТЧИК

150–300К Р

в месяц

ДИЗАЙНЕР

150–300К Р

в месяц

У ВАС ВСЁ ЗА

\$100

24/7. Силлы накапливаются.

*Через год: 30-50 **силлов** = команда из 30 специалистов. Кто собирает сейчас - через год запускает проекты за дни.*

08

МСП и CLI

Подключение внешнего мира: серверы, плагины, терминальные утилиты

Model Context Protocol = *USB-порт* для сервисов.

БЕЗ МСП

Файлы + терминал

С МСП

Google Таблицы, календарь, CRM,
браузер - что угодно

ТАБЛИЦЫ

Google Sheets

38 инструментов.
Таблицы целиком.

БРАУЗЕР

Chrome DevTools

Кликать, заполнять,
парсить,
скриншотить.

ПАРСИНГ

Firecrawl

Продвинутый
парсинг сайтов.

ДОКУМЕНТАЦИЯ

Context7

Актуальная
документация
библиотек.

Установка: `smithery.ai` → копируете промпт → вставляете в Claude Code.

ПОДВОХ

МСП *жрут контекст.*

Каждый МСП добавляет описания **ВСЕХ инструментов в контекст при КАЖДОМ сообщении**, даже если вы сейчас не работаете с ним.

ОДИН МСП GOOGLE SHEETS

~2 000

токенов в каждом сообщении

10 МСП УСТАНОВЛЕНО

20 000+

токенов просто так

МСП

Загрузка: ВСЕГДА. **Контекст:** ~2000+ ток. **Когда:** частое использование.

Скилл

Загрузка: по запросу. **Контекст:** ~100 ток (поиск). **Когда:** редкое использование.

| Лайфхак: «Сделай скилл из МСП Google Sheets, чтобы отключить МСП» - огромная экономия.

ТРИ СПОСОБА ПОДКЛЮЧЕНИЯ МИРА

Плагины *и* CLI-инструменты.

~100 ТОК (ПОИСК)

Скиллы

Инструкции для задач. Лучше для редкого использования.

~2000+ ТОК (ВСЕГДА)

MCP

Частые внешние сервисы. Жрут контекст постоянно.

0 ТОКЕНОВ

CLI

Терминальные программы. **НУЛЕВАЯ нагрузка** на контекст.

Плагины – расширения самого Claude Code. Скилл = инструкция. MCP = сервис. Плагин = новые возможности. Управление: `/plugins` .

CLI: `yt-dlp` (YouTube), `ffmpeg` (видео), `Whisper` (транскрибация), `Puppeteer` (браузер), `gws` (Google).

09

Субагенты и деплой

Параллельные агенты, оркестратор, GitHub → Vercel, телефон, среды

Каждый агент – *в своём контексте.*

ЭКОНОМИЯ КОНТЕКСТА

Файл 50К → выжимка 500 ток.

Субагент анализирует в своём контексте, возвращает 500 токенов выжимки.
Главный агент чист.

ПАРАЛЛЕЛЬНОСТЬ

5 субагентов одновременно

Один исследует конкурентов, другой парсит, третий генерирует отчёт.

Живут в `.claude/agents/`.YAML: `name, description, model, tools`.

Каждому – своя модель: **Opus** (анализ), **Sonnet** (код), **Haiku** (рутина).

План → Фазы → Агенты → *Критик* → Сборка.

01 **Один → много.** Главный агент пишет план, запускает 5 субагентов параллельно. Каждый в своём контексте.

02 **Агент-критик.** Отдельный агент проверяет план на ошибки, находит риски, предлагает правки. Усиливает качество в разы.

03 **Финальная сборка.** Выжимки от субагентов собираются в результат. Главный чат остаётся чистым.

Это не просто «много агентов». Это методология: план → фазы → параллельные субагенты → критик → финал.

Оркестратор *параллельных агентов.*

ПРОМТ-ОРКЕСТРАТОР

«Запусти параллельных субагентов для реализации плана. Перед стартом каждый читает CLAUDE.md, business/, план. Для КАЖДОЙ фазы определи: можно параллелить или нельзя.»

Главное предупреждение: два агента, правящие один файл, **сломают друг друга**. Параллелить можно только непересекающиеся фазы (разные папки / разные модули).

Оркестратор разбивает план на: *сначала 1 блокирующая фаза → потом 3-5 параллельных → финальная сборка. Ускорение в 3-5 раз.*

ДЕПЛОЙ

Claude Code → GitHub → Vercel → сайт в интернете.

«Запуши на GitHub» – код в репозитории. Vercel подхватывает автоматически. **30 секунд** от изменения до обновления на сайте.

REMOTE CONTROL

Приложение Claude (iOS/Android)

QR-код → управляете с телефона. **Условие:** компьютер включён.

TELEGRAM-BOT

Bot Father → плагин → связка

Общаетесь с Claude Code прямо из Telegram.

| *Деплой и доступ подробно разберём на дне 3.*

Расширение Claude *для Google Chrome.*

AI сам **кликает по сайтам, заполняет формы, выгружает данные** – как живой пользователь.

Кейс 1

Создать сервер в Яндекс.Облаке без открытия вкладок

Кейс 2

Настроить CRM, собрать API-ключ

Кейс 3

Выгрузить отчёт из закрытой системы

Просите Claude Code *написать промт* для браузерного агента - он напишет, вы вставите, Chrome-клюд сделает всё сам.

Команды Claude Code.

/INIT

Инициализация, `CLAUDE.md`

/CONTEXT

Контекст и токены

/COMPACT

Сжать историю

/CLEAR

Новый диалог

/MODEL

Сменить модель

/CONFIG

Настройки

/MCP

MCP-серверы

/PLUGINS

Плагины

/REWIND

Откат

/RESUME

Возобновить

/VOICE

Голосовой ввод

/USAGE

Лимиты

Дополнительно: **!** – выполнить терминальную команду. Дальше – **вершина**: всё это собирается в эффект «ИИ работает, пока ты спишь». Триггеры.

10

Triggers

ИИ работает, пока ты спишь

ИИ не ждёт, когда вы зайдёте. *Он сам приходит.*

У ЧЕЛОВЕКА

Проснулся → вспомнил → пошёл делать

У ИИ

Время / условие → запустился → принёс решение

- **По расписанию.** Каждое утро в 7:00.
- **По условию.** Если заявок просело > 10%.
- **По cron.** Каждый понедельник – разбор недели.

7:00 *утра*. Триггер срабатывает сам.

ИИ-клон → берёт `mastery/financial-analysis/`
→ идёт в `business/economics/`
→ смотрит `business/goals/quarterly.md`
→ подключается к прод-базе по **LIVE-маркеру**

Видит: заявки просели на 10%. Математика не сходится.

*В Telegram прилетает: «Сегодня надо **оторвать попу от дивана** и решить вопрос с маркетингом - не дожмёшь квартал».*

Не «сделай маркетинг» - а **вывод эксперта по цифрам**.

ПОД КАПОТОМ

Три кирпичика.

01 · КОГДА ЗАПУСКАЕТСЯ

Cron

Системный планировщик. «Каждый день в 7:00».
Запускает скрипт.

02 · ЧТО ДЕЛАЕТ

Промпт-сценарий

Файл с инструкцией: «иди туда, возьми это,
обработай так, верни вывод сюда».

03 · КУДА ПРИХОДИТ

Канал доставки

Telegram-бот / email / push. Куда придёт результат.

Принцип переносится на любую задачу: конкуренты, отзывы, упавшие метрики, поддержка.

КАНАЛ ДОСТАВКИ

Почему *Telegram*.

Не открываете ноутбук. Не заходите в чат. **ИИ** присылает сам – туда, где вы и так живёте.

1

и и

«Цены конкурентов изменились. Рекомендую X.»

2

в ы

«Да»

3

и и

Идёт и делает.

| Утром, в дороге, перед сном – *управление бизнесом в одно касание.*

СЛЕДУЮЩАЯ СТУПЕНЬ

Один триггер – счастье. Двадцать – новая проблема.

20 триггеров → ИИ-оркестратор → 1 утренняя сводка

Не 20 уведомлений в 7:00. **Один отчёт:** что важно сегодня, чему уделить внимание, что игнорировать.

| *Оркестратор тоже триггер. Просто над триггерами.*

Технология доступна. Большинство не знает, что делать.

Сидят в ChatGPT, спрашивают «напиши пост», и думают, что это ИИ. А вы знаете: **ai-clone, mastery, business, context engineering, триггеры.**

РЫНОК ИИ-АГЕНТОВ СЕЙЧАС

\$8 млрд

ЧЕРЕЗ 2 ГОДА

\$50+ млрд

25% крупных компаний уже внедряют. Через 2 года - 50%.

*Это как интернет в 1998. Кто разобрался - владеет компаниями.
Кто «потом» - работает на кого-то.*

Инструкция на вечер – не «по желанию».

Сегодня – *каркас*. Не полная сборка.

СЕГОДНЯ ДА

- CLAUDE.md, .claude/, plans/ – каркас
- Каркас ai-clone + business
- Одна папка mastery
- Понимание context engineering
- Принципы триггеров
- 2–3 must-have скилла поставлены

СЕГОДНЯ НЕТ

- Полная сборка mastery (30+ методов)
- 30+ узлов business с подузлами
- Полноценные триггеры с cron
- Деплой триггеров на сервер
- Оркестратор поверх 20 триггеров
- Свои собственные скиллы

Победить одну маленькую задачу целиком. *Завтра (День 3) – публикация и автоматизация.*

Простой *vs* профессиональный.

БЕЗ ПОДГОТОВКИ

- Один промпт
- Без скиллов
- Без business/ai-clone
- Без спецификации
- Без референсов

Результат: **нормальный, но обычный, генеричный.**

С МЕТОДОЛОГИЕЙ

- CLAUDE.md + ai-clone + business + mastery
- Skills (Frontend Design, Bulletproof)
- MCP + референсы
- Детальная спецификация
- Цикл итераций + критика

Результат: **как от профессиональной студии.**

Один и тот же Claude. *Разница - в подходе.* Всё, что разобрали сегодня, и есть эта разница.

Инструкция будет *следующая*.

По принципу **копировать и вставить**. Готовые промпты и шаблоны – **выйдут завтра утром**.

- 01 **Создать структуру.** ai-clone/ + business/ + mastery/ + plans/ + retrospectives/
- 02 **Настроить CLAUDE.md.** До 200 строк. Роутинг + 6 правил поведения. /init сгенерирует базу.
- 03 **Поставить Skills.** Frontend Design + Skill Creator + Bulletproof – **обязательно**.
- 04 **Собрать ai-clone** через интервью-промпт. identity + voice + principles. Голосом через /voice .
- 05 **Собрать business** через интервью-промпт. products + audience + economics + goals.
- 06 **Одну папку mastery.** Копирайтинг / переговоры / финанализ – что нужнее.
- 07 **Один план в plans/** . Plan Mode → файл YYYY-MM-DD-название.md . Фазы []/[x] .
- 08 **Тестовый запрос.** «Что у меня по цифрам?» или «Напиши пост в моём стиле». Сравните с тем, что было до.
- 09 **Рефлексия** в retrospectives/ . Что узнали. Что записать в feedback/.

*Цель – маленький, но **живой** второй мозг. Завтра – выводим в интернет и настраиваем триггеры.*

Один Claude. *Разница - в подходе.*

1 **CLAUDE.md + .claude/ + plans/ - фундамент.**
Системный промпт проекта + файловая память + один план = одна функция. Без этого агент - стажёр с нуля каждый раз.

2 **AI-Clone + Mastery + Business - второй мозг.**
Вы в файлах + концентрат экспертов + карта бизнеса. ИИ работает как вы, не вместо вас.

3 **Context Engineering вытесняет prompt engineering.**
Контекст в файлах. Промпт короткий. ИИ сам выбирает что взять под задачу.

4 **Триггеры - ИИ приходит к вам с выводами.**
7:00 утра, финансовый аналитик в Telegram. Не вы идёте к ИИ - он идёт к вам.

5 **Методология вайб-кодинга - 6 шагов.**
Инструкция → Вопросы → CLAUDE.md → Реализация → Итерации → Чанки. Research+Plan+Critique **до** кода.

6 **Skills + MCP + субагенты - команда экспертов.**
\$100/мес = дизайнер + аналитик + копирайтер 24/7. Через год - 30-50 скиллов = 30 специалистов.

СМЫСЛОКОД

Спасибо.

Артеми́й Милле́р · СмыслоКо́д

Соберите `ai-clone` + `business` сегодня вечером.
Завтра - выводим в интернет.